

Deployment Report

`learning-project-java-webapp-deployment-stack`

Purpose of this Document

This document explains the deployment of a Java web application using Maven, Apache Tomcat, and NGINX on an AWS EC2 instance.

The goal of the document is not only to describe the deployment steps, but also to explain the underlying concepts, system structure, component interactions, and troubleshooting process involved in the deployment workflow.

The document focuses on:

- Maven build and packaging concepts
- WAR deployment using Tomcat
- Reverse proxy configuration using NGINX
- Request flow through the deployment stack
- Runtime behavior of JSP applications
- Troubleshooting and debugging steps encountered during deployment

1. High-Level Process Overview

This report demonstrates the deployment of a Java web application using a layered architecture consisting of a build system, an application runtime, and a reverse proxy in an aws instance.

The application is first built using Maven, which compiles the source code and packages it into a WAR (Web Application Archive) file. This WAR file is then deployed on Apache Tomcat, which acts as the application server (application runtime) responsible for executing the Java web application.

To make the application accessible externally in a clean and secure way, NGINX is used as a reverse proxy i.e.. NGINX listens on port 80 and forwards incoming HTTP requests to Tomcat running on port 8080.

The overall flow of the system is as follows:

Client (Browser)

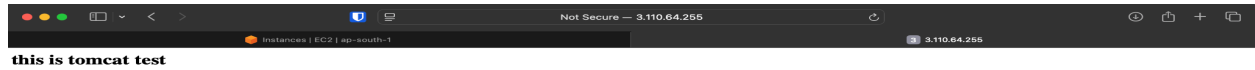
↓

NGINX (port 80)

↓

Tomcat (port 8080)

↓
Java Web Application (WAR)
↓
Response returned to client



2. Concepts

2.1 Compilation and Packaging

A Java web application begins as human-readable source code written in .java files. Before it can run, this code must be transformed into a form that the Java runtime can execute.

Compilation is the process of converting .java files into .class files (bytecode). This is done by the Java compiler (javac). Bytecode is not machine-specific; it is designed to run on the Java Virtual Machine (JVM).

Packaging is the process of grouping compiled code and required resources into a single deployable unit. For web applications, this unit is called a WAR (Web Application Archive).

A WAR file contains:

- compiled classes
- configuration files
- web resources (JSP, HTML, etc.)

This packaging allows the application to be easily deployed on a server like Tomcat without manual file placement.

Note: If Java source files are present, they are compiled into .class files.

In this project, the application primarily uses JSP files, which are compiled into Java classes by Tomcat at runtime.

Process flow:

Project Source Structure

```
|— pom.xml (Maven build configuration)
|— src/
|   |— main/
|       |— java/          (optional: .java files – application logic)
|       |— webapp/        (JSP / HTML / CSS – web resources)
|       |— WEB-INF/
|           |— web.xml (configuration)
```

In a typical Java web application, .java files (if present) are compiled by the Java compiler (javac) into .class files.

However, in this project, no standalone .java files are present. The application primarily uses JSP (JavaServer Pages) files.

JSP files are not compiled during the Maven build process. Instead, they are compiled at runtime by Tomcat:

JSP → converted to .java → compiled to .class → executed.

The web resources and configuration files remain unchanged during the build process.

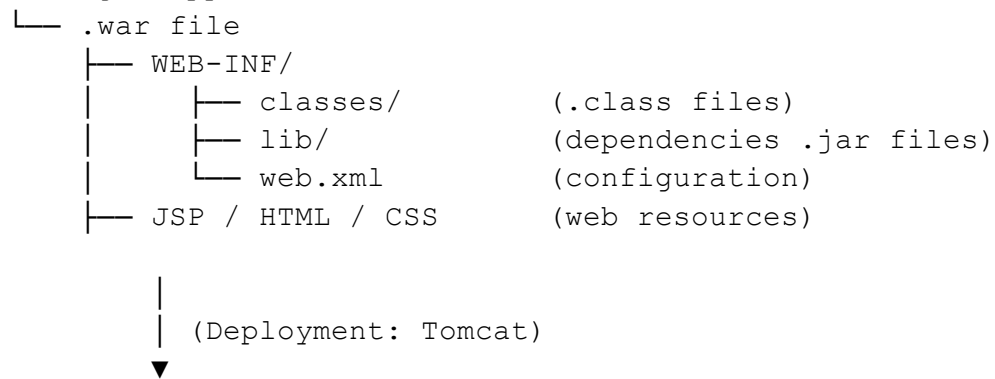
The pom.xml file is used by Maven to define how the project is built, but it is not included in the final packaged application.

During packaging, Maven collects:

- web resources (JSP, HTML, CSS)
- configuration files (web.xml)
- (and compiled .class files if any exist)

and combines them into a WAR (Web Application Archive) file.

Packaged Application



Running Application

2.2 Maven – Why it Exists and What It Does

Before build tools like Maven, building Java applications was a manual and error-prone process.

Developers had to:

- compile source files manually using `javac`
- manage dependencies (external libraries) by downloading JAR files and placing them correctly
- ensure correct build order and classpaths
- package the application manually into deployable formats

This led to several problems:

- inconsistent builds across environments
- missing or incorrect dependencies
- complex and repetitive build steps
- lack of standard project structure

Maven was created to solve these problems by introducing a standardized and automated build system.

Maven uses a central configuration file called `pom.xml` (Project Object Model), which defines:

- project structure
- dependencies
- build process
- plugins and lifecycle

Using Maven, the entire build process can be executed with a single command:

```
# mvn clean package
```

Internally, Maven performs:

- cleaning previous builds
- compiling source code
- resolving and downloading dependencies
- packaging the application into a WAR file

This ensures that builds are:

- repeatable
- consistent across systems
- automated

Without Maven:

javac → manage dependencies manually → manual packaging → error-prone

With Maven:

mvn clean package → automated build → consistent WAR output

2.3 Apache Tomcat – Application Runtime

Apache Tomcat is a Java application server that is responsible for running Java web applications.

After the application is built into a WAR file, it cannot run by itself. A runtime environment is required to:

- accept HTTP requests from clients
- interpret and execute Java web components (Servlets, JSP)
- generate and return HTTP responses

Tomcat provides this runtime environment.

When a WAR file is placed inside Tomcat's webapps directory, Tomcat performs the following steps:

1. Detects the WAR file
2. Extracts it into a directory (called an "exploded" application)
3. Loads the application into memory
4. Starts listening for HTTP requests (default port 8080)

When a client sends a request:

- Tomcat receives the request
- routes it to the appropriate application
- executes the corresponding Java code
- returns the generated response

In this project, the application is deployed as ROOT.war, which means it is served at the root path:

http://<ip>:8080/

Tomcat Directory Structure

```
/opt/tomcat/
├── bin/
├── conf/
├── webapps/
│   ├── ROOT/          (your deployed application - extracted WAR)
│   └── ROOT.war        (your original WAR file)
├── logs/
└── temp/
```

IMPORTANT: What creates ROOT/

ROOT.war → Tomcat extracts it → creates ROOT/

Tomcat Process flow:

WAR file (input)

↓

Placed in /webapps/

↓

Tomcat extracts WAR → creates ROOT/

↓

Application loaded into memory

↓

Tomcat listens on port 8080

↓

Handles HTTP requests → returns response

2.4 NGINX and Reverse Proxy

NGINX is a web server that can also function as a reverse proxy.

In a basic setup, a client (browser) sends a request directly to the application server (Tomcat) on port 8080. However, exposing the application server directly has several limitations:

- non-standard port (8080 instead of 80)
- lack of control over incoming traffic
- reduced flexibility for scaling and security

To solve this, NGINX is placed in front of Tomcat as a reverse proxy.

A reverse proxy is a server that receives client requests and forwards them to one or more backend servers. The client interacts only with the proxy and is unaware of the backend server.

In this project:

- NGINX listens on port 80 (standard HTTP port)
- it receives incoming requests from the client
- it forwards those requests internally to Tomcat running on port 8080
- Tomcat processes the request and returns a response
- NGINX sends the response back to the client

This creates a layered architecture where:

- NGINX handles external traffic
- Tomcat handles application execution

Nginx Process Flow:

```
Client (Browser)
    ↓
NGINX (port 80)
    ↓
Tomcat (port 8080)
    ↓
Java Application
    ↓
Response returned to client
```

NGINX acts as an intermediary that exposes the application on port 80 while forwarding requests internally to Tomcat.

3. Structural Views of the System

Java Project Structure

```
Project Root
├── pom.xml                (build configuration)
├── src/
│   ├── main/
│   │   ├── java/          (.java source files)
│   │   ├── webapp/         (JSP / HTML / CSS)
│   │   └── WEB-INF/
│   │       └── web.xml      (application configuration)
```

MAVEN Build Structure (Output)

```
target/
└─ java-tomcat-maven-example.war
```

TOMCAT Directory Structure

```
/opt/tomcat/
├─ bin/           (startup/shutdown scripts)
├─ conf/          (server configuration)
├─ webapps/
│   └─ ROOT.war   (deployed WAR file)
│       └─ ROOT/   (extracted application)
├─ logs/          (runtime logs)
└─ temp/          (temporary files)
```

Nginx Configuration Structure

```
/etc/nginx/
├─ nginx.conf     (main configuration file)
├─ conf.d/
│   └─ tomcat.conf (reverse proxy configuration)
```

Nginx config hierarchy

```
main
├─ http
│   └─ server
│       └─ location
```

Each component in the system has a defined structure: the project structure organizes source code, Maven defines the build output, Tomcat manages deployment directories, and NGINX organizes request handling through hierarchical configuration.

4. End-to-End Workflow (Concept to Execution Mapping)

4.1 – Build the Application (Detailed)

Step 1: Build the Java Web Application using Maven

Command:


```
# cd /home/ec2-user/java_proj_sample-main/  
# mvn clean package
```

```
# cd /home/ec2-user/java_proj_sample-main/  
- Changes directory to the project root  
- This is required because Maven needs access to `pom.xml`
```

```
# mvn clean package  
- Executes the Maven build lifecycle
```

```
plexus-cipher-1.8-3.amzn2023.0.3.noarch  
plexus-classworlds-2.6.0-10.amzn2023.0.4.noarch  
plexus-containers-component-annotations-2.1.0-9.amzn2023.0.4.noarch  
plexus-interpolation-1.26-10.amzn2023.0.4.noarch  
plexus-sec-dispatcher-2.0-3.amzn2023.0.3.noarch  
plexus-utils-3.3.0-9.amzn2023.0.5.noarch  
publicsuffix-list-20260116-1.amzn2023.0.1.noarch  
sisu-1:0.3.4-9.amzn2023.0.4.noarch  
slf4j-1.7.32-3.amzn2023.0.4.noarch  
  
Complete!  
[root@ip-172-31-23-185 ec2-user]#  
[root@ip-172-31-23-185 ec2-user]#  
[root@ip-172-31-23-185 ec2-user]#  
[root@ip-172-31-23-185 ec2-user]# cd java_proj_sample-main/  
[root@ip-172-31-23-185 java_proj_sample-main]# ls  
Jenkinsfile  Procfile  README.md  pom.xml  src  
[root@ip-172-31-23-185 java_proj_sample-main]# mvn clean package  
[INFO] Scanning for projects...  
[INFO]  
[INFO] -----< com.example:java-tomcat-maven-example >-----  
[INFO] Building hello Maven Webapp 1.0-SNAPSHOT  
[INFO] -----[ war ]-----  
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom  
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom (3.9 kB at 20 kB/s)  
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom  
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom (13 kB at 334 kB/s)  
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom  
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom (26 kB at 613 kB/s)
```

```

-artifact-transfer-0.13.1.jar (159 kB at 393 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-collections/commons-collections/3.2.2/commons-collections-3.2.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-lang3/3.12.0/commons-lang3-3.12.0.jar (587 kB at 1.4 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-logging/commons-logging/1.2/commons-logging-1.2.jar (62 kB at 144 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-collections4/4.2/commons-collections4-4.2.jar (753 kB at 1.7 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-beanutils/commons-beanutils/1.9.4/commons-beanutils-1.9.4.jar (247 kB at 560 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/commons-collections/commons-collections/3.2.2/commons-collections-3.2.2.jar (588 kB at 1.3 MB/s)
[INFO] Configured Artifact: com.github.jsimone:webapp-runner:8.5.11.3:jar
Downloading from central: https://repo.maven.apache.org/maven2/com/github/jsimone/webapp-runner/8.5.11.3/webapp-runner-8.5.11.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/jsimone/webapp-runner/8.5.11.3/webapp-runner-8.5.11.3.pom (5.5 kB at 36 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/github/jsimone/webapp-runner-parent/8.5.11.3/webapp-runner-parent-8.5.11.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/jsimone/webapp-runner-parent/8.5.11.3/webapp-runner-parent-8.5.11.3.pom (12 kB at 55 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/com/github/jsimone/webapp-runner/8.5.11.3/webapp-runner-8.5.11.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/jsimone/webapp-runner/8.5.11.3/webapp-runner-8.5.11.3.jar (8.2 MB at 21 MB/s)
[INFO] Copying webapp-runner-8.5.11.3.jar to /home/ec2-user/java_proj_sample-main/target/dependency/webapp-runner.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 19.224 s
[INFO] Finished at: 2026-05-05T05:48:30Z
[INFO] -----

```

Output of this step

```

target/
└─ java-tomcat-maven-example.war

```

```

[root@ip-172-31-23-185 java_proj_sample-main]# ls
Jenkinsfile Procfile README.md pom.xml src target
[root@ip-172-31-23-185 java_proj_sample-main]# ls target/
dependency java-tomcat-maven-example java-tomcat-maven-example.war maven-archiver

```

This WAR file is ready to be deployed to Tomcat.

Maven automates the entire build process, converting source code into a deployable WAR file using a standardized lifecycle.

4.2 – Install and Prepare Tomcat (Detailed)

Step 2: Install and prepare Apache Tomcat (application server)

Why Tomcat is needed

A WAR file cannot run by itself.

Tomcat provides:

- HTTP request handling
- execution environment for Java web applications
- lifecycle management of deployed applications

Why Tomcat is NOT to be installed using a package manager

Tomcat could have been installed using a package manager such as:

```
# dnf install tomcat
```

However, this approach was NOT used for the following reasons:

- Package managers often provide older or distribution-specific versions
- The installed version may not match project compatibility requirements
- Less control over installation location and configuration
- Real-world deployments often require manual version control

Therefore, Tomcat was installed manually to:

- ensure a known and consistent version
- have full control over installation directory (/opt/tomcat)
- avoid dependency on OS package versions

How the Tomcat version was chosen

The project does not explicitly specify a required Tomcat version in pom.xml.

- The pom.xml file references webapp-runner version 8.5.11.3, indicating that the project was built around an older Tomcat-compatible runtime environment.
- To maintain compatibility with the project tooling and avoid potential runtime issues, Tomcat 9 was selected for deployment.

```

</dependencies>

<build>
  <finalName>java-tomcat-maven-example</finalName>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-dependency-plugin</artifactId>
      <version>3.3.0</version>
      <executions>
        <execution>
          <phase>package</phase>
          <goals><goal>copy</goal></goals>
          <configuration>
            <artifactItems>
              <artifactItem>
                <groupId>com.github.jsimone</groupId>
                <artifactId>webapp-runner</artifactId>
                <version>8.5.11.3</version>
                <destFileName>webapp-runner.jar</destFileName>
              </artifactItem>
            </artifactItems>
          </configuration>
        </execution>
      </executions>
    </plugin>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-war-plugin</artifactId>
      <version>3.3.0</version>
    </plugin>
  </plugins>
</build>

```

This means it is compatible with:

- Tomcat 9 (Servlet 4.0)
- NOT Tomcat 10 (which uses Jakarta namespace changes)

To ensure compatibility, the latest available Tomcat 9 version was selected.

Step-by-step Commands

```
# cd /opt
```

- Move to /opt directory (standard location for optional software)

```
# curl --silent https://downloads.apache.org/tomcat/tomcat-9/ \
| grep --only-matching --extended-regexp 'v9\.0\.[0-9]+' \
| sort --version-sort \
| tail --lines=1
```

- Fetches the latest available Tomcat 9 version dynamically
- Prevents errors caused by hardcoding outdated versions (404 errors)

```
[root@ip-172-31-23-185 java_proj_sample-main]# cd /opt
[root@ip-172-31-23-185 opt]# wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat-9.0.89.tar.gz
--2026-05-05 06:10:38-- https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat-9.0.89.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 95.216.224.44, 2a01:4f8:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-05-05 06:10:38 ERROR 404: Not Found.

[root@ip-172-31-23-185 opt]# wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
--2026-05-05 06:12:24-- https://downloads.apache.org/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 95.216.224.44, 88.99.208.237, 2a01:4f9:2b:1cc2::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|95.216.224.44|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-05-05 06:12:25 ERROR 404: Not Found.

[root@ip-172-31-23-185 opt]# curl --silent https://downloads.apache.org/tomcat/tomcat-9/ \
| grep --only-matching --extended-regexp 'v9\.0\.[0-9]+' \
| sort --version-sort \
| tail --lines=1
v9.0.117
```

#wget

```
https://downloads.apache.org/tomcat/tomcat-9/v9.0.117/bin/apache-tomcat-9.0.117.tar.gz
```

- Downloads Tomcat archive from official Apache source

```
[root@ip-172-31-23-185 java_proj_sample-main]# cd /opt
[root@ip-172-31-23-185 opt]# wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat-9.0.89.tar.gz
--2026-05-05 06:10:38-- https://downloads.apache.org/tomcat/tomcat-9/v9.0.89/bin/apache-tomcat-9.0.89.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 95.216.224.44, 2a01:4f8:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-05-05 06:10:38 ERROR 404: Not Found.

[root@ip-172-31-23-185 opt]# wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
--2026-05-05 06:12:24-- https://downloads.apache.org/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 95.216.224.44, 88.99.208.237, 2a01:4f9:2b:1cc2::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|95.216.224.44|:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-05-05 06:12:25 ERROR 404: Not Found.

[root@ip-172-31-23-185 opt]# curl --silent https://downloads.apache.org/tomcat/tomcat-9/ \
| grep --only-matching --extended-regexp 'v9\.0\.[0-9]+' \
| sort --version-sort \
| tail --lines=1
v9.0.117
[root@ip-172-31-23-185 opt]# wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.117/bin/apache-tomcat-9.0.117.tar.gz
--2026-05-05 06:14:07-- https://downloads.apache.org/tomcat/tomcat-9/v9.0.117/bin/apache-tomcat-9.0.117.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 95.216.224.44, 88.99.208.237, 2a01:4f9:2b:1cc2::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|95.216.224.44|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13089813 (12M) [application/x-gzip]
Saving to: 'apache-tomcat-9.0.117.tar.gz'

apache-tomcat-9.0.117.tar.gz  100%[=====>] 12.48M  5.84MB/s  in 2.1s

2026-05-05 06:14:10 (5.84 MB/s) - 'apache-tomcat-9.0.117.tar.gz' saved [13089813/13089813]
```



```
# tar --extract --gzip --file apache-tomcat-9.0.117.tar.gz
```

- Extracts the downloaded archive
- Creates directory: apache-tomcat-9.0.117/

```
[root@ip-172-31-23-185 opt]# tar --extract --gzip --file apache-tomcat-9.0.117.tar.gz
[root@ip-172-31-23-185 opt]# ls
apache-tomcat-9.0.117  apache-tomcat-9.0.117.tar.gz  aws
[root@ip-172-31-23-185 opt]# mv apache-tomcat-9.0.117 tomcat
[root@ip-172-31-23-185 opt]# ls
apache-tomcat-9.0.117.tar.gz  aws  tomcat
```

```
# mv apache-tomcat-9.0.117 tomcat
```

- Renames directory for simplicity
- Final path becomes: /opt/tomcat

What happened so far

1. Files are downloaded from Apache repository
2. Archive is extracted into a directory
3. Tomcat directory structure is created
4. Startup and shutdown scripts become available

Resulting Directory Structure

```
/opt/tomcat/
├── bin/           (startup/shutdown scripts)
├── conf/          (server configuration)
├── webapps/       (applications will be deployed here)
├── logs/          (runtime logs)
└── temp/          (temporary files)
```

Tomcat was manually installed to ensure compatibility, version control, and proper runtime support for the Java web application.

4.3 – Deploy WAR File to Tomcat (Detailed)

Step 3: Deploy the built application (WAR file) to Tomcat

Why deployment is required

After building the application using Maven, the output is a WAR file.

This WAR file must be placed inside Tomcat so that:

- Tomcat can detect it

- extract it
- load it as a runnable application

Step-by-step Commands

```
# cp /home/ec2-user/java_proj_sample-main/target/java-tomcat-maven-example.war /opt/tomcat/webapps/ROOT.war
```

- Copies the WAR file into Tomcat's deployment directory
- Renames the file to ROOT.war during the copy operation

```
[root@ip-172-31-23-185 opt]# ls /opt/tomcat/
BUILDING.txt  LICENSE  README.md  RUNNING.txt  conf  logs  webapps
CONTRIBUTING.md  NOTICE  RELEASE-NOTES  bin  lib  temp  work
[root@ip-172-31-23-185 opt]# cp /home/ec2-user/java_proj_sample-main/target/java-tomcat-maven-example.war /opt/tomcat/webapps/ROOT.war
```

What this command does (important)

cp syntax:

cp <source> <destination>

Here:

- source = the MAVEN built WAR file
- destination = ROOT.war inside Tomcat webapps

Note: This performs TWO actions:

1. Copies the file
2. Renames it to ROOT.war

Why rename to ROOT.war

Tomcat uses the file name to determine the URL path.

Rules:

- app.war → accessible at /app
- ROOT.war → accessible at /

So:

If no rename done then

java-tomcat-maven-example.war will be mapped to -> /java-tomcat-maven-example

But, since we renamed the file to ROOT.war

ROOT.war will be mapped to → /

This ensures the application is available at:

http://<ip>:8080/

What happens internally (after deployment)

When Tomcat starts:

1. It scans the /opt/tomcat/webapps directory
2. Detects ROOT.war
3. Extracts it into a directory:

/opt/tomcat/webapps/ROOT/

4. Loads the application into memory
5. Maps it to the root URL (/)

Resulting structure

```
/opt/tomcat/webapps/  
├── ROOT.war    (input WAR file)  
└── ROOT/      (extracted application directory)
```

Renaming the WAR file to ROOT.war ensures the application is deployed at the root URL and accessible directly without a path.

4.4 – Start Tomcat and Load Application (Detailed)

Step 4: Start the Tomcat server and load the deployed application

Step-by-step Commands

```
# /opt/tomcat/bin/startup.sh
```

- Starts the Tomcat server
- Initializes the application runtime environment


```
[root@ip-172-31-23-185 opt]# /opt/tomcat/bin/startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
```

What this command does

The script:

- sets environment variables (CATALINA_HOME, CATALINA_BASE)
- initializes Java runtime
- starts the Tomcat process in the background

What happens internally (step-by-step)

1. Tomcat process starts
2. Reads configuration files from:
 /opt/tomcat/conf/
3. Scans deployment directory:
 /opt/tomcat/webapps/
4. Detects:
 ROOT.war
5. Extracts WAR file into:
 /opt/tomcat/webapps/ROOT/
6. Loads application classes into memory
7. Initializes servlets and JSP engine
8. Starts listening on port:

 8080 (default HTTP port for Tomcat)

Important behavior not note

If BOTH exist:

/opt/tomcat/webapps/ROOT/
and
/opt/tomcat/webapps/ROOT.war

Then:

Tomcat will use the ROOT/ directory (already extracted version)

NOT the ROOT.war file

So make sure to remove any existing ROOT/ before starting tomcat

Why this matters

If ROOT/ contains old/default files:

- your new application will NOT load
- Tomcat will serve the old existing defaults ROOT/ content

Result of successful startup

Tomcat prints:

"Tomcat started."

And becomes accessible at:

http://localhost:8080

Tomcat starts the runtime environment, extracts the WAR file, and loads the application to serve requests on port 8080.

4.5 – Verify Application on Port 8080 (Detailed)

Step 5: Verify that the application is running correctly on Tomcat

Step-by-step Commands

```
# curl http://localhost:8080
```

- Sends an HTTP request to the local Tomcat server
- Used to test application response from within the EC2 instance

```
[root@ip-172-31-23-185 opt]# curl http://localhost:8080
<html>
<body>
<h2>this is tomcat test</h2>
</body>
</html>
```

What this command does

curl:

- acts as a command-line HTTP client
- sends a request to the specified URL
- prints the server response in the terminal

What happens internally

1. Request is sent to:
localhost:8080
2. Tomcat receives the request
3. Maps request to the ROOT application
4. Executes application code
5. Generates response
6. Returns response to curl

Expected output (successful case)

You should see:

```
<h2>this is tomcat test</h2>
```

This confirms:

- application is deployed correctly
- Tomcat is running
- request-response cycle is working

What this step verifies

- Tomcat is active
- WAR file has been correctly deployed
- Application is functioning internally
- Port 8080 is accessible locally

The curl command confirms that Tomcat is successfully running the deployed application on port 8080.

4.6 – Install and Start NGINX (Detailed)

Step 6: Install NGINX to act as a reverse proxy

Why NGINX is needed

Accessing the application directly via:

http://<ip>:8080

has limitations:

- non-standard port (8080)
- exposes application server directly
- lacks control over incoming traffic

NGINX solves this by:

- listening on port 80 (standard HTTP port)
- acting as an entry point
- forwarding requests to Tomcat internally

Step-by-step Commands

```
# dnf install nginx --assumeyes
```

- Installs NGINX from the system repository
- Automatically confirms installation prompts

```
# systemctl start nginx
```

- Starts the NGINX service
- NGINX begins listening on port 80

```
# systemctl enable nginx
```

- Enables NGINX to start automatically on system boot

NGINX is installed and started to act as the public-facing entry point for incoming HTTP requests on port 80.

4.7 – Configure NGINX Reverse Proxy (Detailed)

Step 7: Configure NGINX to forward requests to Tomcat

Why configuration is required

By default, NGINX serves static HTML from:

```
/usr/share/nginx/html
```

This means:

- requests to port 80 do NOT reach Tomcat
- only the default NGINX page is shown

To connect NGINX with Tomcat, a reverse proxy configuration is required.

Step-by-step Commands

```
# vi /etc/nginx/conf.d/tomcat.conf
```

- Creates a new configuration file for our application
- Keeps configuration modular and separate from main config

Add the following configuration

```
server {  
    listen 80;  
  
    location / {  
        proxy_pass http://127.0.0.1:8080;  
    }  
}
```

What this configuration does

server block:

- defines how NGINX handles incoming requests

listen 80:

- NGINX listens on port 80 (HTTP)

location /:

- matches all incoming requests

proxy_pass http://127.0.0.1:8080:

- forwards requests to Tomcat running locally on port 8080

```
#vi /etc/nginx/nginx.conf
```

Comment out the default configuration i.e everything inside server {...}

```

        "$http_user_agent" "$http_x_forwarded_for";

access_log /var/log/nginx/access.log main;

sendfile            on;
tcp_nopush          on;
keepalive_timeout   65;
types_hash_max_size 4096;

include             /etc/nginx/mime.types;
default_type        application/octet-stream;

# Load modular configuration files from the /etc/nginx/conf.d directory.
# See http://nginx.org/en/docs/nginx_core_module.html#include
# for more information.
include /etc/nginx/conf.d/*.conf;

# server {
#     listen      80;
#     listen      [::]:80;
#     server_name _;
#     root        /usr/share/nginx/html;

#     # Load configuration files for the default server block.
#     include /etc/nginx/default.d/*.conf;

#     error_page 404 /404.html;
#     location = /404.html {
#     }

#     error_page 500 502 503 504 /50x.html;
#     location = /50x.html {
#     }
}

```

47,2

38%

Apply configuration

```
# nginx -t
```

- Tests configuration for syntax errors

```
# systemctl restart nginx
```

- Restarts NGINX to apply changes

What happens internally

1. NGINX starts listening on port 80
2. Client sends request to NGINX
3. NGINX receives request
4. Matches it to location /
5. Forwards request to:

127.0.0.1:8080 (Tomcat)

6. Tomcat processes request
7. Response sent back to NGINX
8. NGINX returns response to client

NGINX forwards incoming HTTP requests on port 80 to Tomcat on port 8080, enabling external access without exposing the application server directly.

4.8 – Access Application via Public IP (Detailed)

Step 8: Access the application externally through NGINX

Why this step is important

So far:

- Tomcat is running on port 8080 (internal)
- NGINX is configured on port 80 (external entry point)

This step verifies:

- full end-to-end connectivity
- reverse proxy is working correctly
- application is accessible from outside the server

Requirement: Security Group Configuration

To allow external access, the EC2 security group must permit HTTP traffic.

Ensure the following rule exists:

Type: HTTP

Port: 80

Source: 0.0.0.0/0

This allows incoming traffic from the internet to reach NGINX.

Edit inbound rules [Info](#)

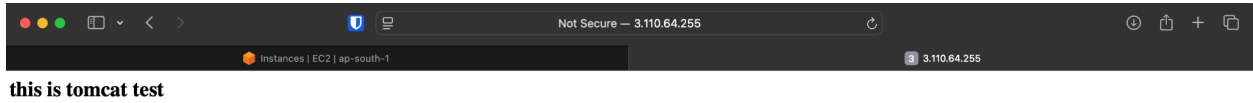
Inbound rules control the incoming traffic that's allowed to reach the instance.

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
sgr-0da151e6b8ce8a839	SSH ModifyInboundSecurityGroupRules	TCP	22	Custom Q	0.0.0.0/0 X	Delete
sgr-03aa08ba0426fbc1e	HTTP	TCP	80	Custom Q	0.0.0.0/0 X	Delete

Access from browser

Open a browser and navigate to:

http://<public-ip>



What happens internally

1. Browser sends request to public IP on port 80
2. Request reaches EC2 instance
3. Security group allows the request
4. NGINX receives request on port 80
5. NGINX forwards request to Tomcat (127.0.0.1:8080)
6. Tomcat processes request
7. Application generates response
8. Response flows back:
Tomcat → NGINX → Browser

Expected output

You should see:

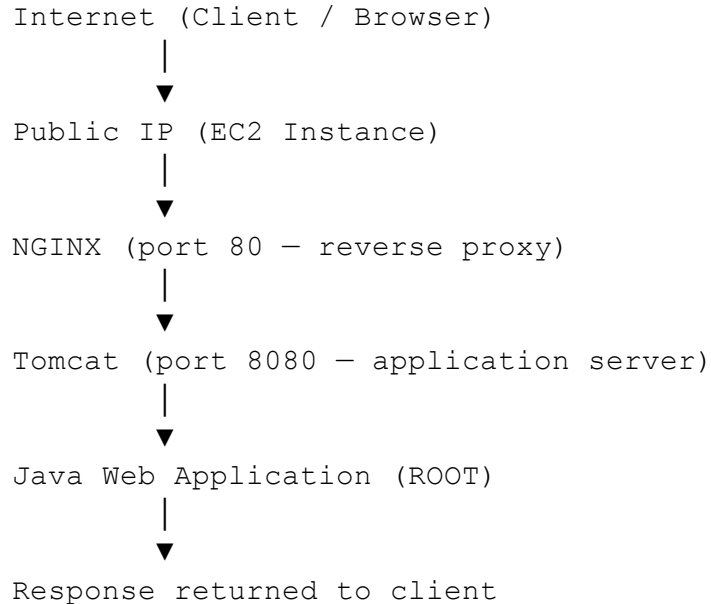
this is tomcat test

This confirms:

- NGINX reverse proxy is working
- Tomcat is running correctly
- Application is accessible externally

The application is successfully exposed to the internet through NGINX, completing the full deployment pipeline.

5 – Final Architecture and Request Flow



Layered Responsibility

Each component in the system has a specific role:

NGINX:

- handles incoming HTTP requests
- acts as a reverse proxy
- forwards requests to backend server

Tomcat:

- executes Java web application
- processes business logic
- generates HTTP responses

Application:

- contains user-facing functionality
- returns content (HTML/JSP output)

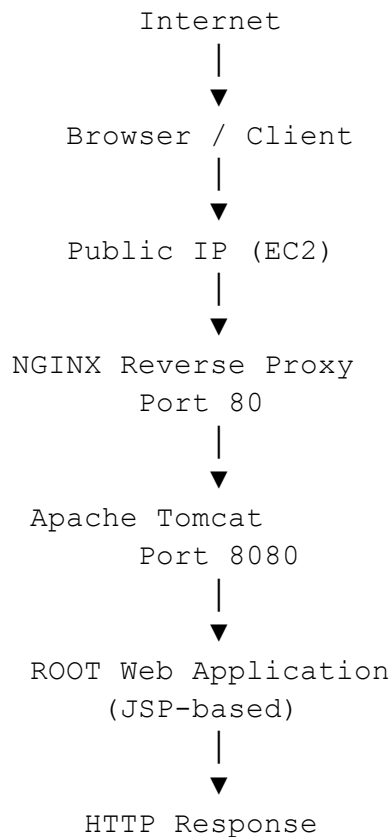
Request Flow (Step-by-step)

1. Client sends request to:
http://<public-ip>

2. Request reaches EC2 instance on port 80
3. NGINX receives the request
4. NGINX forwards request to:
127.0.0.1:8080
5. Tomcat receives the request
6. Tomcat routes request to ROOT application
7. Application executes and generates response
8. Response flows back:
Application → Tomcat → NGINX → Client

The deployed system uses a layered architecture where NGINX exposes the application externally while Tomcat executes it internally, ensuring clean separation and controlled access.

6 - Final Architecture Summary Diagram



The browser communicates only with NGINX on port 80.
NGINX forwards requests internally to Tomcat on port 8080, where the Java web application is executed.

This architecture separates:

- external traffic handling
- application execution
- deployment logic

and is commonly used in production environments.

7 - Key Learning Outcomes

This project provided practical and conceptual understanding of how a Java web application is built, deployed, and exposed through a layered architecture.

Maven and Build Systems

Learned:

- the role of Maven in automating build workflows
- the difference between compilation and packaging
- how pom.xml defines project structure and build behavior
- how WAR files are generated for deployment

JSP Runtime Behavior

Learned:

- JSP files are compiled at runtime by Tomcat
- runtime compilation flow:
JSP → .java → .class → executed
- difference between build-time processing and runtime processing

Apache Tomcat

Learned:

- Tomcat acts as the Java application runtime environment
- WAR deployment lifecycle:
deploy → extract → load → execute
- how ROOT.war maps to the root URL (/)
- Tomcat deployment behavior using exploded directories

NGINX and Reverse Proxy

Learned:

- difference between web server and application server
- how reverse proxy architecture works
- request forwarding from port 80 → 8080
- layered separation between traffic handling and application execution

Linux and System Administration

Learned:

- manual software installation using archives
- service management using systemctl
- configuration management using NGINX config files
- directory structure organization in Linux environments

Networking and Cloud Concepts

Learned:

- role of EC2 security groups as cloud firewalls
- difference between localhost access and external access
- request flow from browser → NGINX → Tomcat → application

Troubleshooting and Debugging

Learned:

- diagnosing deployment conflicts
- understanding configuration hierarchy errors
- interpreting service startup failures
- debugging reverse proxy behavior
- verifying compatibility between software components

Overall Understanding

This project demonstrated how multiple independent components:

- Maven
- Tomcat
- NGINX
- Linux
- AWS networking

work together to form a complete application deployment pipeline.

8 – Conclusion

This report demonstrates the complete lifecycle of deploying a Java web application from source code to a publicly accessible service.

9 - Troubleshooting and Debugging

Issue 1 – Default Tomcat Page Displayed Instead of Application

Problem:

After deploying the WAR file, Tomcat continued showing the default page instead of the deployed application.

Cause:

The directory:

```
/opt/tomcat/webapps/ROOT/
```

already existed.

Tomcat prioritizes the extracted ROOT/ directory over ROOT.war.

Fix:

```
# rm --recursive --force /opt/tomcat/webapps/ROOT
```

Then restart Tomcat.

Concept Learned:

Tomcat deployment behavior uses the exploded application directory if it already exists.

Issue 2 – NGINX Configuration Error

Problem:

NGINX failed to start and returned:

```
"location directive is not allowed here"
```

Cause:

The location block was placed outside the server block.

Incorrect hierarchy:

```
http → location
```

Correct hierarchy:

```
http → server → location
```

Fix:

Move the location block inside the server block.

Concept Learned:

NGINX configuration follows a strict hierarchical structure.

Issue 3 – NGINX Default Welcome Page Still Appearing

Problem:

Requests to port 80 continued displaying the default NGINX page.

Cause:

The default server configuration inside nginx.conf was still active and overriding the custom reverse proxy configuration.

Fix:

Comment out the default server block and restart NGINX.

Concept Learned:

NGINX processes requests using matching server blocks, and default configurations can override custom configurations.

Issue 4 – Tomcat Download Returned 404 Error

Problem:

Attempting to download Tomcat using a hardcoded version returned:

404 Not Found

Cause:

The specified version no longer existed on Apache download servers.

Fix:

Use a dynamic version lookup command:

```
# curl --silent https://downloads.apache.org/tomcat/tomcat-9/ \  
| grep --only-matching --extended-regexp 'v9\.0\.[0-9]+' \  
| sort --version-sort \  
| tail --lines=1
```

Concept Learned:

Software versions should be verified dynamically rather than hardcoded.

Issue 5 – Application Not Accessible from Browser

Problem:

Application worked locally but was inaccessible externally.

Cause:

The EC2 security group did not allow inbound HTTP traffic on port 80.

Fix:

Add inbound rule:

Type: HTTP

Port: 80

Source: 0.0.0.0/0

Concept Learned:

Cloud firewall rules control whether external traffic can reach services running inside an instance.